



جامعة هواري بومدين للعلوم والتكنولوجيا
The University of Science and Technology Houari Boumediene

Faculty of Computer Science

Chahinez Benaicha

MINI PROJECT RAPPORT

Introduction

This report presents the implementation of a Mini-Project on Multi-Agent Systems (MAS) and it's structured into four distinct parts:

- **Part 1** — Multi-Agent Negotiation: An auction system with 1 seller and multiple buyer agents.
- **Part 2** — Multi-Criteria Decision & Mobile Agents: A buyer agent that migrates across containers to collect and evaluate multi-criteria offers.
- **Part 3** — AIMA Partial Order Planner: Implementation and testing of the Partial Order Planning (POP) algorithm from the AIMA textbook.
- **Part 4** — Centralized Planning for Distributed Plans: A central planner that decomposes a global goal and distributes sub-plans to agents.

All JADE-based parts (1 & 2) are implemented in Java using the JADE framework. Parts 3 and 4 are implemented in Python.

Part 1 — Multi-Agent Negotiation: Auction System

1.1 Problem Description

The objective of Part 1 is to simulate an English auction (ascending-price) between one seller agent and a set of buyer agents using the JADE (Java Agent DEvelopment) framework.

The auction protocol proceeds as follows:

- The seller puts a product up for sale with an opening price.
- Buyers offer amounts higher than the current asking price.
- The seller identifies the highest bid and broadcasts it to all buyers.
- Rounds repeat until all buyers stop bidding or the maximum round limit is reached.
- If the final highest price exceeds the seller's secret reserve price, the item is sold to the winner.

1.2 Architecture & Agents

Agent	Role	Key Behaviours
SellerAgent	Manages auction rounds; announces prices; selects winner	WakerBehaviour, AuctionBehaviour (state machine)
BuyerAgent	Receives CFPs; submits bids within budget; reacts to outcome	CyclicBehaviour (AuctionParticipantBehaviour)
AuctionLauncher	Bootstraps JADE platform; creates all agents	Main class

1.6 Numerical Example

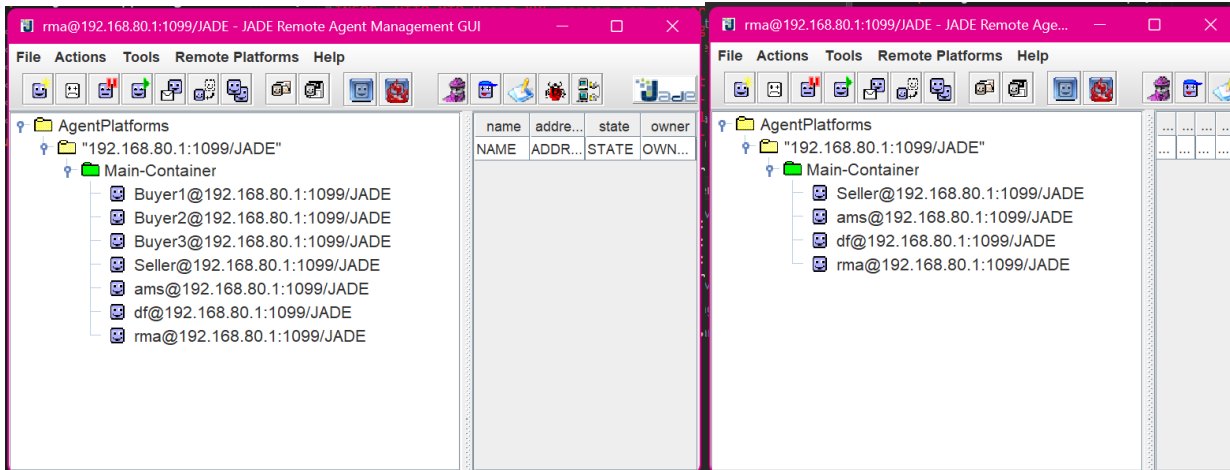
Three buyers with the following configurations:

Agent	Budget	Increment
Buyer1	750.00	100.00
Buyer2	820.00	105.00
Buyer3	680.00	110.00

With opening price 500 and reserve price 700, Buyer2 (highest budget, largest increment) is expected to win (after several rounds of competitive bidding).

Avant la vente:

Après la vente :



```
All agents started. Auction will begin shortly...
[Seller] Found 3 buyer(s).
```

```
[Seller] — Round 1 — Current price: 500,00
[Buyer2] Bidding 605,00 (budget: 820,00)
[Buyer1] Bidding 600,00 (budget: 750,00)
[Buyer3] Bidding 610,00 (budget: 680,00)
[Seller] Bid from Buyer2: 605,00
[Seller] Bid from Buyer1: 600,00
[Seller] Bid from Buyer3: 610,00
[Seller] Best bid so far: 610,00 by Buyer3
```

```
[Seller] — Round 2 — Current price: 610,00
[Buyer3] Passing — 720,00 exceeds budget 680,00
[Buyer2] Bidding 715,00 (budget: 820,00)
[Buyer1] Bidding 710,00 (budget: 750,00)
[Seller] Bid from Buyer2: 715,00
[Seller] Bid from Buyer1: 710,00
[Seller] Best bid so far: 715,00 by Buyer2
```

```
[Seller] — Round 3 — Current price: 715,00
[Buyer3] Passing — 825,00 exceeds budget 680,00
[Buyer1] Passing — 815,00 exceeds budget 750,00
[Buyer2] Bidding 820,00 (budget: 820,00)
[Seller] Bid from Buyer2: 820,00
[Seller] Best bid so far: 820,00 by Buyer2
[Seller] Max rounds reached.
```

```
SOLD: Tableau Les femmes d'alger by Pablo Picasso
Winner : Buyer2
Price : 820,00
```

```
[Buyer3] Did not win.
[Buyer1] Did not win.
```

```
[Buyer2] *** WON the auction! Product: Tableau Les femmes d'alger by Pablo Picasso Price: 820.0 ***
```

Part 2 — Multi-Criteria Decision & Mobile Agents

2.1 Problem Description (Exercise 5 — DW4)

A buyer agent seeks to purchase product x from multiple competing sellers. Each seller's offer is characterised by a set of criteria $C_i(x)$, for which the buyer has defined preference weights $P_i(x)$. Some criteria must be maximised (quality), others minimised (price, delivery cost).

The buyer must evaluate all offers and select the best one using a systematic multi-criteria scoring formula. Additionally, the buyer is implemented as a mobile agent that physically migrates to each seller's container to collect the offer.

2.2 Multi-Criteria Formula $f(x)$

2.2.1 Formalisation

Let there be n sellers, each offering product x with m criteria:

- $C_1(x)$ = Price — to MINIMISE — weight P_1
- $C_2(x)$ = Quality — to MAXIMISE — weight P_2
- $C_3(x)$ = Delivery Cost — to MINIMISE — weight P_3

Constraint: $P_1 + P_2 + P_3 = 1$

2.2.2 Normalisation Step

To make criteria comparable, each is normalised to $[0, 1]$:

For MAXIMISE criteria (quality):

$$\text{norm}_i(x) = (C_i(x) - C_{i_min}) / (C_{i_max} - C_{i_min})$$

For MINIMISE criteria (price, delivery):

$$\text{norm}_i(x) = (C_{i_max} - C_i(x)) / (C_{i_max} - C_{i_min})$$

Edge case: if $C_{i_max} = C_{i_min}$ (all sellers equal on criterion i), then $\text{norm}_i(x) = 1.0$ for all.

2.2.3 Aggregation Formula

$$f(x) = P_1 * \text{norm_price}(x) + P_2 * \text{norm_quality}(x) + P_3 * \text{norm_delivery}(x)$$

The best offer is: $x^* = \text{argmax } f(x)$

2.3 Numerical Example

Three sellers with the following offers:

Seller	Price C_1	Quality C_2	Delivery C_3
Seller A	450	85	30
Seller B	380	60	15
Seller C	520	92	25

Weights: P_1 (price) = 0.5, P_2 (quality) = 0.3, P_3 (delivery) = 0.2

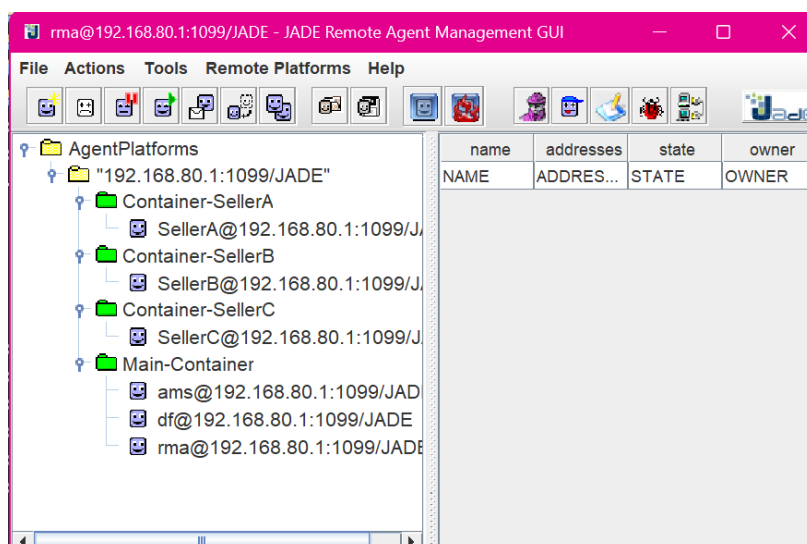
2.4 Mobile Agent Architecture

The MobileBuyerAgent uses JADE's agent mobility (doMove()) to migrate between containers:

Container	Agent	Role
Main-Container	MobileBuyerAgent (home)	Starts here; evaluates here
Container-SellerA	SellerAgent A	Responds to offer requests
Container-SellerB	SellerAgent B	Responds to offer requests
Container-SellerC	SellerAgent C	Responds to offer requests

Migration sequence:

- Buyer starts in Main-Container, discovers sellers via DF.
- Buyer migrates to Container-SellerA → sends REQUEST → receives INFORM with offer.
- Buyer migrates to Container-SellerB → collects offer.
- Buyer migrates to Container-SellerC → collects offer.
- Buyer migrates back to Main-Container → evaluates all offers → selects best.



Started SellerA in Container-SellerA
[SellerA] Offer – Price: 450,00 Quality: 85,00 Delivery: 30,00

Started SellerB in Container-SellerB
[SellerB] Offer – Price: 380,00 Quality: 60,00 Delivery: 15,00

Started SellerC in Container-SellerC
[SellerC] Offer – Price: 520,00 Quality: 92,00 Delivery: 25,00
MobileBuyerAgent started. It will now migrate to each seller...
=== MOBILE BUYER AGENT STARTED ===
Weights – Price: 0,50 Quality: 0,30 Delivery: 0,20
Found seller: SellerC
Found seller: SellerB
Found seller: SellerA
[MobileBuyer] Migrating to: Container-SellerC
[MobileBuyer] Arrived at: Container-SellerC
[SellerC] Sent offer to MobileBuyer
[MobileBuyer] Collected from SellerC → P=520,00 Q=92,00 D=25,00
[MobileBuyer] Migrating to: Container-SellerB
[SellerC] Sent offer to MobileBuyer
[MobileBuyer] Arrived at: Container-SellerB
[MobileBuyer] Collected from SellerB → P=520,00 Q=92,00 D=25,00
[MobileBuyer] Migrating to: Container-SellerA
[SellerB] Sent offer to MobileBuyer
[MobileBuyer] Arrived at: Container-SellerA
[MobileBuyer] Collected from SellerA → P=380,00 Q=60,00 D=15,00
[SellerA] Sent offer to MobileBuyer
[MobileBuyer] All sellers visited. Returning home...
[MobileBuyer] Arrived at: Main-Container

MULTI-CRITERIA EVALUATION				
Seller	Price	Quality	Delivery	f(x)
SellerC	520,00	92,00	25,00	0,3000
SellerB	520,00	92,00	25,00	0,3000
SellerA	380,00	60,00	15,00	0,7000

Weights: P1(price)=0,50 P2(quality)=0,30 P3(delivery)=0,20

```
BEST OFFER: SellerA
Score f(x) = 0,7000
Price=380,00 Quality=60,00 Del=15,00
```

Part 3 — AIMA Partial Order Planner

3.1 Theoretical Background

A Partial Order Plan (POP) represents a planning solution where actions are not fully sequenced — only the ordering constraints necessary for correctness are imposed. This provides flexibility and supports parallel execution.

3.2 POP Algorithm

A POP consists of:

- Steps: a set of actions (plus special Start and Finish steps)
- Ordering constraints: $S_i < S_j$ means S_i must be executed before S_j
- Causal links: $S_i \dashrightarrow[p] S_j$ means S_i achieves precondition p needed by S_j
- Open preconditions: preconditions not yet supported by any causal link

The Partial Order Planning algorithm:

- Base case: if no open preconditions exist, return the complete plan.
- Select an open precondition $\langle p, S_j \rangle$ from the agenda.
- For each action S_i (existing or new) that achieves p : add causal link $S_i \dashrightarrow[p] S_j$, add ordering $S_i < S_j$.
- If S_i is new: add it to the plan and add its preconditions to the open set.
- Resolve all threats: for each action S_k that might delete p , apply demotion ($S_k < S_i$) or promotion ($S_j < S_k$).
- Recurse.

3.3 Sussman Anomaly Results

The classic Sussman Anomaly (stacking blocks A, B, C) was solved. The planner correctly identified the need to unstack C from A before placing B on C:

Step	Action	Description
0	Start	Initial state: On(C,A), On(A,Table), On(B,Table), Clear(C), Clear(B)
1	MoveToTable(C,A)	Move block C from A to Table → clears A
2	Move(B,Table,C)	Move block B from Table to C → On(B,C) achieved
3+	...	Continue to achieve On(A,B)
N	Finish	Goal: On(A,B), On(B,C) achieved


```

— SOLUTION —
Total order (one valid linearisation):
0. Start
1. MoveToTable(C,A)

Causal links:
(Move(B,Table,C) --[On(B,C)]--> Finish)
(Move(A,C,B) --[On(A,B)]--> Finish)
(Start --[Clear(C)]--> Move(B,Table,C))
(Start --[On(B,Table)]--> Move(B,Table,C))
(Start --[Clear(B)]--> Move(B,Table,C))
(MoveToTable(C,A) --[Clear(A)]--> Move(A,C,B))
(Move(A,B,C) --[On(A,C)]--> Move(A,C,B))
(Start --[Clear(B)]--> Move(A,C,B))
(Start --[Clear(C)]--> MoveToTable(C,A))
(Start --[On(C,A)]--> MoveToTable(C,A))
(Start --[Clear(C)]--> Move(A,B,C))
(MoveToTable(C,A) --[Clear(A)]--> Move(A,B,C))
(Move(A,C,B) --[On(A,B)]--> Move(A,B,C))

```

```

Ordering constraints:
Move(A,B,C) < Move(B,Table,C)
Move(A,B,C) < Finish
Move(A,B,C) < Move(A,C,B)
Move(A,C,B) < Move(A,B,C)
Move(A,C,B) < Finish
Move(B,Table,C) < Move(A,C,B)
Move(B,Table,C) < Finish
MoveToTable(C,A) < Move(A,C,B)
MoveToTable(C,A) < Move(B,Table,C)
MoveToTable(C,A) < Finish
MoveToTable(C,A) < Move(A,B,C)
Start < Move(B,Table,C)
Start < Move(A,C,B)
Start < MoveToTable(C,A)
Start < Finish
Start < Move(A,B,C)

```

3.4 Spare Tyre Results

The Spare Tyre problem was solved in 3 steps: Remove(Spare,Boot), Remove(Flat,Axle), PutOn(Spare,Axle). No threats were detected — the two removal actions are independent and could be parallelised.

```

Initial state : {'At(Flat,Axle)', 'At(Spare,Boot)', 'Intact(Spare)'}
Goal state    : {'At(Spare,Axle)'}

Open precondition: 'At(Spare,Axle)' needed by Finish
Trying to satisfy 'At(Spare,Axle)' with PutOn(Spare,Axle)
Open precondition: 'Clear(Axle)' needed by PutOn(Spare,Axle)
Trying to satisfy 'Clear(Axle)' with Remove(Flat,Axle)
Open precondition: 'At(Spare,Ground)' needed by PutOn(Spare,Axle)
Trying to satisfy 'At(Spare,Ground)' with Remove(Spare,Boot)
Open precondition: 'At(Flat,Axle)' needed by Remove(Flat,Axle)
Trying to satisfy 'At(Flat,Axle)' with Start
Open precondition: 'At(Spare,Boot)' needed by Remove(Spare,Boot)
Trying to satisfy 'At(Spare,Boot)' with Start
✓ Plan complete!

```

```

— SOLUTION —
0. Start
1. Remove(Spare,Boot)
2. Remove(Flat,Axle)
3. PutOn(Spare,Axle)
4. Finish

Causal links:
(PutOn(Spare,Axle) --[At(Spare,Axle)]--> Finish)
(Remove(Flat,Axle) --[Clear(Axle)]--> PutOn(Spare,Axle))
(Remove(Spare,Boot) --[At(Spare,Ground)]--> PutOn(Spare,Axle))
(Start --[At(Flat,Axle)]--> Remove(Flat,Axle))
(Start --[At(Spare,Boot)]--> Remove(Spare,Boot))

```

Part 4 — Centralized Planning for Distributed Plans

4.1 Concept

In multi-agent planning, a central authority (the CentralPlanner) constructs a global plan and decomposes it into per-agent sub-plans. This approach ensures global coherence while enabling distributed execution.

4.2 Architecture

Component	Role
CentralPlanner	Global STRIPS planning; decomposition; sync identification
AgentExecutor	Executes assigned sub-plan; updates shared world state
SyncPoint	Constraint: agent A must complete action X before agent B starts Y
Action (STRIPS)	Defined by: name, agent, preconditions, add-effects, del-effects

4.3 Example 1: Logistics Domain

Two trucks deliver packages to a destination city:

- Truck1: Load(Pkg1) → Drive(CityA,CityC) → Unload(Pkg1)
- Truck2: Load(Pkg2) → Drive(CityB,CityC) → Unload(Pkg2)

The two sub-plans are fully independent (no synchronisation needed). Both goal conditions At(Pkg1,CityC) and At(Pkg2,CityC) are achieved.

```
EXAMPLE 1: Logistics Domain

CENTRAL PLANNER - Decomposition

Initial state : ['At(Pkg1,CityA)', 'At(Pkg2,CityB)', 'At(Truck1,CityA)', 'At(Truck2,CityB)', 'Road(CityA,CityC)', 'Road(CityB,CityC)']
Goal          : ['At(Pkg1,CityC)', 'At(Pkg2,CityC)']
Agents        : ['Truck1', 'Truck2']

--- Global Plan ---
1. [Truck1] Load(Pkg1,Truck1,CityA)
2. [Truck1] Drive(Truck1,CityA,CityC)
3. [Truck1] Unload(Pkg1,Truck1,CityC)
4. [Truck2] Load(Pkg2,Truck2,CityB)
5. [Truck2] Drive(Truck2,CityB,CityC)
6. [Truck2] Unload(Pkg2,Truck2,CityC)

--- Sub-plans by Agent ---
Truck1:
• Load(Pkg1,Truck1,CityA)
• Drive(Truck1,CityA,CityC)
• Unload(Pkg1,Truck1,CityC)
Truck2:
• Load(Pkg2,Truck2,CityB)
• Drive(Truck2,CityB,CityC)
• Unload(Pkg2,Truck2,CityC)

--- Synchronisation Points ---
(none - agents are independent)

--- Distributed Execution ---

[Truck1] Starting execution...
[Truck1] Executing: Load(Pkg1,Truck1,CityA)
[Truck1] ✓ State after: ['At(Pkg2,CityB)', 'At(Truck1,CityA)', 'At(Truck2,CityB)', 'In(Pkg1,Truck1)', 'Road(CityA,CityC)', 'Road(CityB,CityC)']
[Truck1] Executing: Drive(Truck1,CityA,CityC)
[Truck1] ✓ State after: ['At(Pkg2,CityB)', 'At(Truck1,CityC)', 'At(Truck2,CityB)', 'In(Pkg1,Truck1)', 'Road(CityA,CityC)', 'Road(CityB,CityC)']
[Truck1] Executing: Unload(Pkg1,Truck1,CityC)
[Truck1] ✓ State after: ['At(Pkg1,CityC)', 'At(Pkg2,CityB)', 'At(Truck1,CityC)', 'At(Truck2,CityB)', 'Road(CityA,CityC)', 'Road(CityB,CityC)']
[Truck1] Sub-plan complete.

[Truck2] Starting execution...
[Truck2] Executing: Load(Pkg2,Truck2,CityB)
[Truck2] ✓ State after: ['At(Pkg1,CityC)', 'At(Truck1,CityC)', 'At(Truck2,CityB)', 'In(Pkg2,Truck2)', 'Road(CityA,CityC)', 'Road(CityB,CityC)']
[Truck2] Executing: Drive(Truck2,CityB,CityC)
[Truck2] ✓ State after: ['At(Pkg1,CityC)', 'At(Truck1,CityC)', 'At(Truck2,CityC)', 'In(Pkg2,Truck2)', 'Road(CityA,CityC)', 'Road(CityB,CityC)']
[Truck2] Executing: Unload(Pkg2,Truck2,CityC)
[Truck2] ✓ State after: ['At(Pkg1,CityC)', 'At(Pkg2,CityC)', 'At(Truck1,CityC)', 'At(Truck2,CityC)', 'Road(CityA,CityC)', 'Road(CityB,CityC)']
[Truck2] Sub-plan complete.

Final world state: ['At(Pkg1,CityC)', 'At(Pkg2,CityC)', 'At(Truck1,CityC)', 'At(Truck2,CityC)', 'Road(CityA,CityC)', 'Road(CityB,CityC)']
Goal achieved: ✓ YES
```

4.4 Example 2: Rescue Mission

Two robots must cooperate to rescue a victim from a locked room:

- Robot1: Navigate to Room2 → Open Door
- Robot2: Navigate to Room2 (needs door open!) → Pickup Victim → Carry to SafeZone

The planner identifies a synchronisation point:

SYNC: Robot1.OpenDoor(Room2) → Robot2.Navigate2_to_Room2

Robot2 cannot navigate to Room2 until Robot1 has opened the door. This dependency is automatically detected from the causal structure: OpenDoor adds DoorOpen(Room2), which is a precondition of Navigate2_to_Room2.

```
EXAMPLE 2: Rescue Mission (MAS Chapter 3 style)

CENTRAL PLANNER – Decomposition

Initial state : ['At(Robot1,Entrance)', 'At(Robot2,Entrance)', 'At(Victim,Room2)', 'DoorClosed(Room2)']
Goal          : ['At(Victim,SafeZone)']
Agents       : ['Robot1', 'Robot2']

— Global Plan —
1. [Robot1] Navigate1_to_Room2
2. [Robot1] OpenDoor(Room2)
3. [Robot2] Navigate2_to_Room2
4. [Robot2] PickupVictim
5. [Robot2] CarryToSafeZone

— Sub-plans by Agent —
Robot1:
• Navigate1_to_Room2
• OpenDoor(Room2)
Robot2:
• Navigate2_to_Room2
• PickupVictim
• CarryToSafeZone

— Synchronisation Points —
SYNC: Robot1.OpenDoor(Room2) → Robot2.Navigate2_to_Room2

— Distributed Execution —

[Robot1] Starting execution...
[Robot1] Executing: Navigate1_to_Room2
[Robot1] ✓ State after: ['At(Robot1,Room2)', 'At(Robot2,Entrance)', 'At(Victim,Room2)', 'DoorClosed(Room2)']
[Robot1] Executing: OpenDoor(Room2)
[Robot1] ✓ State after: ['At(Robot1,Room2)', 'At(Robot2,Entrance)', 'At(Victim,Room2)', 'DoorOpen(Room2)']
[Robot1] Sub-plan complete.

[Robot2] Starting execution...
[Robot2] Executing: Navigate2_to_Room2
[Robot2] ✓ State after: ['At(Robot1,Room2)', 'At(Robot2,Room2)', 'At(Victim,Room2)', 'DoorOpen(Room2)']
[Robot2] Executing: PickupVictim
[Robot2] ✓ State after: ['At(Robot1,Room2)', 'At(Robot2,Room2)', 'Carrying(Robot2,Victim)', 'DoorOpen(Room2)']
[Robot2] Executing: CarryToSafeZone
[Robot2] ✓ State after: ['At(Robot1,Room2)', 'At(Robot2,SafeZone)', 'At(Victim,SafeZone)', 'DoorOpen(Room2)']
[Robot2] Sub-plan complete.

Final world state: ['At(Robot1,Room2)', 'At(Robot2,SafeZone)', 'At(Victim,SafeZone)', 'DoorOpen(Room2)']
Goal achieved: ✓ YES
```

Conclusion

This mini-project covered four complementary aspects of Multi-Agent Systems:

- Part 1 demonstrated agent negotiation through a complete English auction protocol using JADE's ACL messaging and DF service discovery. The key insight is that auction protocols can be cleanly modelled as communicating state machines.
- Part 2 introduced multi-criteria decision-making with the weighted normalised scoring formula $f(x)$, and extended it to a mobile agent scenario where the buyer physically migrates to each seller. The formula correctly handles both maximisation and minimisation criteria through appropriate normalisation.
- Part 3 implemented the Partial Order Planner from AIMA, successfully resolving the Sussman Anomaly and spare tyre problem.
- Part 4 showed how a central planner can decompose a global goal into independent agent sub-plans, automatically identifying the synchronisation points needed for inter-agent coordination.